

- Documentation Projet BioGuard Access
  - 1. Présentation générale
  - 2. Problème adressé
  - 3. Objectifs du projet
  - 4. Périmètre fonctionnel actuel
    - 4.1 Côté Raspberry Pi
    - 4.2 Côté mobile
  - 5. Public cible
  - 6. Proposition de valeur
  - 7. Vue d'ensemble de l'architecture
  - 8. Description des sous-systèmes
    - 8.1 Backend IoT
    - 8.2 Application mobile
    - 8.3 Stockage local et cloud
  - 9. Capteurs et actionneurs
    - 9.1 Capteurs utilisés
    - 9.2 Actionneurs utilisés
  - 10. Pipeline biométrique actuel
    - 10.1 Étapes principales
    - 10.2 Pourquoi ce choix
    - 10.3 Compatibilité et traces auxiliaires
  - 11. Flux métier principaux
    - 11.1 Enrôlement
    - 11.2 Identification / accès
    - 11.3 Réglages et supervision
  - 12. Technologies utilisées
    - 12.1 Backend
    - 12.2 Mobile
  - 13. Structure générale du dépôt
    - 13.1 Dossier iot/
    - 13.2 Dossier Mobile/
    - 13.3 Dossier docs/
  - 14. Données manipulées
  - 15. Sécurité et stratégie d'accès
  - 16. Valeur académique et démonstrative
  - 17. Limites connues
    - 17.1 Limites matérielles

- [17.2 Limites biométriques](#)
- [17.3 Limites d'intégration](#)
- [18. Coût approximatif du prototype](#)
- [19. Positionnement économique simplifié](#)
- [20. Conclusion](#)

# Documentation Projet BioGuard Access

---

## 1. Présentation générale

---

BioGuard Access est un prototype de contrôle d'accès connecté construit autour d'un Raspberry Pi, d'une application mobile Expo et d'une chaîne de communication MQTT. Le système vise à démontrer qu'un contrôle d'accès biométrique peut être réalisé avec une architecture simple, explicable et relativement abordable, tout en conservant :

- une exécution locale côté objet connecté
- une supervision distante sur mobile
- une persistance locale en cas de coupure réseau
- une synchronisation cloud quand les services distants sont disponibles

Le projet n'est pas une simple interface de démonstration. Il contient :

- une logique matérielle réelle
- un backend IoT opérationnel
- un pipeline biométrique déterministe
- une application mobile fonctionnelle
- une base locale SQLite et une synchronisation Firestore

## 2. Problème adressé

---

De nombreuses petites structures ont besoin d'un contrôle d'accès plus robuste qu'un code PIN, mais n'ont ni le budget ni l'infrastructure pour déployer une solution industrielle complète. Les problèmes courants sont :

- partage ou divulgation d'un code d'accès
- manque de traçabilité des entrées et sorties

- dépendance à un PC ou à un serveur central
- matériel trop coûteux pour un prototype académique ou une petite structure

BioGuard Access cherche à répondre à ce besoin avec un compromis entre simplicité, coût, démonstrabilité et niveau de sécurité raisonnable pour un prototype.

## 3. Objectifs du projet

---

Les objectifs techniques et pédagogiques du projet sont les suivants :

1. construire un système embarqué capable de piloter du matériel réel
2. relier ce système à une application mobile utilisable par un opérateur
3. documenter clairement les échanges de données entre le mobile et le backend
4. intégrer une biométrie explicable sans recourir à un gros modèle IA opaque
5. conserver une capacité de fonctionnement local grâce à SQLite
6. exposer une synchronisation cloud avec Firebase pour la consultation distante

## 4. Périmètre fonctionnel actuel

---

Le prototype actuel couvre les fonctions suivantes.

### 4.1 Côté Raspberry Pi

- capture d'image via caméra Pi ou trame simulée en mode mock
- lecture du capteur de lumière
- allumage automatique ou manuel des LED d'éclairage
- pilotage LED verte, LED rouge, buzzer et écran LCD I2C
- enrôlement biométrique multi-captures
- scan d'identification
- journalisation locale des accès, audits et états du dispositif
- publication de télémétrie et réponse aux commandes MQTT

### 4.2 Côté mobile

- connexion et création de compte via Firebase Authentication

- mémorisation optionnelle de la session
- configuration de l'adresse du Raspberry Pi et des ports MQTT
- consultation de l'état du système
- gestion des utilisateurs
- lancement d'un enrôlement
- lancement d'un scan d'accès
- consultation de l'historique d'accès et des audits
- ajustement des préférences utilisateur et de certains réglages matériels

## 5. Public cible

---

Le système s'adresse surtout à des environnements où la démonstration, la traçabilité et l'intégration embarquée comptent davantage qu'une industrialisation complète.

Exemples de cibles plausibles :

- PME
- laboratoires pédagogiques
- salles de matériel ou de serveurs
- bureaux administratifs
- clubs techniques ou makerspaces
- projets universitaires de sécurité ou d'IoT

## 6. Proposition de valeur

---

Le projet se distingue par une combinaison de choix pragmatiques.

- Le traitement biométrique est local et explicable.
- La supervision mobile est simple à démontrer.
- Le backend ne dépend pas d'une API HTTP lourde.
- Le système continue à fonctionner localement même si Firebase est coupé.
- Le code est suffisamment structuré pour être maintenable dans un cadre académique.

## 7. Vue d'ensemble de l'architecture

---

Application mobile Expo / React Native

- | - Auth Firebase
- | - Stores Zustand
- | - Client MQTT over WebSocket
- | - Consultation et administration

v

Broker MQTT

v

Backend IoT Raspberry Pi

- | - mqtt\_gateway.py
- | - core/security\_controller.py
- | - biometrics/biometrics\_service.py
- | - database.py
- | - cloud/firebase\_service.py
- | - hardware/\*

v

Matériel local

- | - Caméra Pi
- | - Capteur de lumière
- | - LCD I2C
- | - Buzzer
- | - LED rouge / verte
- | - 2 LED d'éclairage

Stockages

- | - SQLite local
- | - Firebase / Firestore

Le projet est donc composé de trois axes principaux :

- exécution locale sur le Pi
- communication temps réel par MQTT
- persistance et consultation cloud via Firebase

## 8. Description des sous-systèmes

### 8.1 Backend IoT

Le backend IoT correspond au dossier **iot/** et constitue le cœur temps réel du système.

Responsabilités :

- initialiser la base SQLite
- démarrer la boucle MQTT

- lire les commandes reçues
- piloter le matériel
- produire de la télémétrie
- construire les profils biométriques
- comparer les captures en scan avec les profils enregistrés
- synchroniser une partie des données vers Firebase

## 8.2 Application mobile

L'application mobile correspond au dossier [Mobile/](#).

Responsabilités :

- authentifier l'utilisateur
- maintenir la configuration du point d'accès MQTT
- offrir les écrans de supervision et d'administration
- publier des commandes vers le backend
- recevoir des réponses et de la télémétrie
- synchroniser ou mettre en cache certaines données cloud côté mobile

## 8.3 Stockage local et cloud

Le système combine :

- [SQLite](#) côté Pi pour la résilience locale
- [Firestore](#) pour la visualisation distante et la synchronisation applicative

Cette approche est importante dans un contexte IoT, car elle évite qu'une panne réseau bloque complètement le prototype.

# 9. Capteurs et actionneurs

---

## 9.1 Capteurs utilisés

Le prototype actuel utilise réellement deux types de capteurs.

1. Caméra Raspberry Pi

- utilisée pour la capture biométrique de la main et de la paume
- peut fonctionner en mode réel ou en mode mock selon l'environnement

## 2. Capteur de lumière

- utilisé pour estimer l'obscurité ambiante
- permet d'activer automatiquement les LED d'appoint

## 9.2 Actionneurs utilisés

- LED verte pour le succès d'authentification
- LED rouge pour le refus d'accès
- deux LED d'éclairage pour améliorer les conditions de capture
- buzzer pour le feedback sonore
- écran LCD I2C 16x2 pour l'instruction locale et l'état du système

# 10. Pipeline biométrique actuel

---

Le pipeline biométrique a été recentré sur une logique de type **PalmCode**.

## 10.1 Étapes principales

1. acquisition de l'image via la caméra
2. conversion en niveaux de gris et prétraitement local
3. segmentation de la main
4. détection de points anatomiques de vallée entre les doigts
5. alignement et normalisation d'une ROI palmaire
6. amélioration des lignes de paume
7. filtrage multi-orientation par noyaux de Gabor
8. découpage en anneaux concentriques sur la ROI
9. calcul, pour chaque orientation et chaque anneau, de la moyenne et de la variance
10. concaténation de ces statistiques dans un vecteur **PalmCode**
11. comparaison par similarité cosinus avec les profils enregistrés

## 10.2 Pourquoi ce choix

Ce choix convient bien à un Raspberry Pi parce qu'il est :

- déterministe
- relativement léger
- facile à expliquer devant un jury
- indépendant d'un entraînement complexe

## 10.3 Compatibilité et traces auxiliaires

Le profil biométrique conserve également des informations auxiliaires :

- géométrie globale de la main
- mesures de qualité de capture
- descripteurs de forme conservés pour l'affichage et la compatibilité avec des profils plus anciens

Le cœur du matching moderne est toutefois le vecteur **PalmCode**.

# 11. Flux métier principaux

---

## 11.1 Enrôlement

1. Un administrateur saisit les informations d'un utilisateur dans l'application mobile.
2. Le mobile envoie une commande MQTT d'enrôlement.
3. Le Pi active la prévisualisation, les lumières d'appoint et le guidage LCD.
4. Plusieurs captures sont prises.
5. Les échantillons valides sont fusionnés en un profil biométrique d'enrôlement.
6. Le profil est enregistré dans SQLite.
7. Le profil utilisateur et le profil biométrique sont synchronisés dans Firebase si disponible.
8. Le mobile reçoit le résultat, les identifiants, la télémétrie et les images de debug éventuelles.

## 11.2 Identification / accès



1. Le mobile déclenche un scan ou l'utilisateur s'aligne devant le capteur selon le scénario de démo.
2. Le Pi capture la main, génère un profil live et vérifie la qualité de la capture.
3. Si un `user_id` est fourni, le système compare au profil déclaré.
4. Sinon, il recherche le meilleur candidat parmi les profils stockés localement.
5. Le backend déclenche les actionneurs selon la décision.
6. L'événement est journalisé localement, publié en MQTT et synchronisé vers Firebase si possible.

## 11.3 Réglages et supervision

Le mobile peut :

- modifier le mode d'éclairage
- piloter les LED
- déclencher un test buzzer
- écrire du texte sur le LCD
- ajuster le seuil d'obscurité
- consulter la télémétrie matérielle

## 12. Technologies utilisées

---

### 12.1 Backend

- Python 3
- `paho-mqtt`
- `opencv-python-headless`
- `numpy`
- `firebase-admin`
- `gpiozero`
- `picamera2`
- RPLCD
- `smbus2`
- Werkzeug

### 12.2 Mobile

- Expo
- React Native
- React Navigation
- Zustand
- Firebase JS SDK
- mqtt
- AsyncStorage
- SecureStore
- i18next

## 13. Structure générale du dépôt

---

### 13.1 Dossier **iot/**

- `mqtt_gateway.py` : point d'entrée principal MQTT
- `core/security_controller.py` : orchestration du matériel et des captures
- `biometrics/biometrics_service.py` : extraction biométrique et matching
- `database.py` : persistance SQLite
- `cloud/firebase_service.py` : synchronisation Firestore
- `hardware/` : drivers des capteurs et actionneurs
- `config.py` : configuration matérielle, MQTT, caméra et Firebase

### 13.2 Dossier **Mobile/**

- `App.js` : bootstrap de l'application
- `navigation/NavigationRoot.js` : navigation principale
- `store/authStore.js` : session et préférences utilisateur
- `store/mqttStore.js` : connexion MQTT et requêtes/réponses
- `services/firebase.js` : initialisation Firebase
- `services/cloudSync.js` : synchronisation Firestore côté mobile
- `ecrans/` : écrans fonctionnels de l'application

### 13.3 Dossier **docs/**

Contient la présente documentation structurée.

# 14. Données manipulées

---

Le projet manipule plusieurs familles de données :

- profils utilisateurs
- profils biométriques
- événements d'accès
- journaux d'audit
- télémétrie de l'appareil
- préférences utilisateur côté mobile

Le contrat de détail de ces données est documenté dans [04\\_MQTT\\_Et\\_Donnees.md](#).

# 15. Sécurité et stratégie d'accès

---

Le prototype met en avant plusieurs niveaux de contrôle.

- Authentification des opérateurs via Firebase côté mobile
- Authentification MQTT par identifiant / mot de passe côté broker
- Persistance locale pour éviter la perte totale de service en cas de panne cloud
- Journalisation des opérations d'administration et des événements d'accès

Il faut néanmoins garder en tête que le projet reste un prototype académique.

Ce qui n'est pas couvert comme dans un produit industriel :

- chiffrement bout en bout applicatif spécifique au domaine biométrique
- HSM ou enclave sécurisée
- signature applicative des commandes
- rotation automatisée des secrets
- mécanisme de révocation complexe des gabarits biométriques

# 16. Valeur académique et démonstrative

---

Le projet est fort pédagogiquement parce qu'il relie dans un seul système :

- électronique embarquée

- communication réseau temps réel
- application mobile
- stockage cloud
- traitement d'image
- journalisation locale

Il permet aussi de montrer une vraie frontière de responsabilités entre mobile et objet connecté.

## 17. Limites connues

---

Les limites doivent être assumées clairement.

### 17.1 Limites matérielles

- le prototype ne dispose pas de cinq capteurs différents
- les capteurs effectivement présents sont principalement la caméra et le capteur de lumière

### 17.2 Limites biométriques

- la qualité dépend fortement du cadrage, de l'alignement et de l'éclairage
- le seuil de similarité PalmCode doit être calibré sur des captures réelles
- le système n'est pas conçu comme une biométrie certifiée ou durcie pour production

### 17.3 Limites d'intégration

- le mobile dépend du broker MQTT en WebSocket
- Firebase n'est pas utilisé comme backend HTTP complet, mais comme service d'authentification et de synchronisation
- la cohérence temps réel complète entre SQLite, Firestore et le mobile reste volontairement simple

## 18. Coût approximatif du prototype

---

| Composant                 | Coût estimé   |
|---------------------------|---------------|
| Raspberry Pi              | 90 \$         |
| Caméra Pi                 | 30 \$         |
| Capteur de lumière        | 3 \$          |
| LCD I2C                   | 10 \$         |
| LEDs, résistances, buzzer | 8 \$          |
| Boîtier / intégration     | 20 \$         |
| Alimentation et câblage   | 15 \$         |
| <b>Total estimé</b>       | <b>176 \$</b> |

## 19. Positionnement économique simplifié

---

À l'échelle d'un projet académique, la valeur économique présentable repose moins sur une marge réelle que sur un raisonnement produit :

- solution plus riche qu'un simple badge ou code PIN
- architecture démontrable sur matériel accessible
- supervision mobile moderne
- possibilité d'évolution vers un produit plus complet

## 20. Conclusion

---

BioGuard Access est un prototype cohérent de contrôle d'accès embarqué mêlant :

- matériel réel
- logique IoT
- application mobile
- synchronisation cloud
- biométrie locale explicable

Sa principale force n'est pas d'être un produit industriel fini, mais d'être un système de bout en bout :

- compréhensible
- démontrable
- maintenable
- suffisamment réaliste pour une soutenance technique sérieuse

Les documents spécialisés du dossier [docs/](#) détaillent maintenant chaque aspect du système avec un niveau de précision adapté au développement, à l'installation et à la présentation.